

Kinematic Calibration: Least-Squares Formulation

AR4 Calibration — `calibrationConvergenceSimulation.py`

June 14, 2026

Abstract

This note documents the mathematics of the iterative kinematic calibration used in `calibrationConvergenceSimulation.py`. The procedure is an iterative (Gauss–Newton) weighted linear least-squares estimator: at each iteration it linearizes the forward-kinematics measurement model about the current parameter estimate, forms a stacked residual and Jacobian over all samples, solves the resulting overdetermined linear system by SVD, and applies a damped, clipped, accumulated parameter update. The active parameter set consists of the joint-angle offsets and the base/camera extrinsic offset (12 parameters total).

Note on link lengths. The code declares a set of link-length symbols $l_1, \dots, l_6$, but the line that would multiply them into the kinematic chain (`symbolic_matrix[:3,3] = symbolic_matrix[:3,3] **self.l[i-1]`, line 188) is *commented out*. The forward kinematics therefore never depends on the link lengths, their Jacobian columns are identically zero, and the corresponding estimates remain at machine-epsilon zero. Because they contribute nothing, the link-length parameters are omitted from the formulation below.

1 Parameter vector

The unknown is a correction vector $\boldsymbol{\theta} \in \mathbb{R}^{12}$ partitioned into two physical groups:

$$\boldsymbol{\theta} = \left[\begin{array}{c} \Delta q_1 \\ \Delta q_2 \\ \vdots \\ \Delta q_6 \\ \Delta x_1 \\ \Delta x_2 \\ \vdots \\ \Delta x_6 \end{array} \right] \left\{ \begin{array}{l} \Delta Q \text{ (joint-angle offsets, 6)} \\ \Delta X \text{ (base/camera offset, 6)} \end{array} \right. \quad (1)$$

- ΔQ (6): per-joint angular zero offsets.
- ΔX (6): world→base (or world→camera) extrinsic offset, two translation and three rotation components, with the z translation pinned to 0.

2 Measurement model (forward kinematics)

The kinematic chain is assembled symbolically as a product of 4×4 homogeneous transforms (lines 189–202). In camera mode it predicts the world-to-target pose:

$$T_{\text{origin} \rightarrow \text{target}}(\boldsymbol{\theta}, \mathbf{c}) = T_{\text{origin} \rightarrow \text{base}}(\mathbf{x}) \left(\prod_{i=1}^6 T_i(q_i) \right) T_{\text{camera} \rightarrow \text{target}}(\mathbf{c}), \quad (2)$$

where $\mathbf{c} \in \mathbb{R}^6$ is the measured camera-to-marker pose. The predicted pose is read off this matrix as

$$\mathbf{t}(\boldsymbol{\theta}) = T_{\text{origin} \rightarrow \text{target}}[0:3, 3] \in \mathbb{R}^3 \quad (\text{translation}), \quad (3)$$

$$R(\boldsymbol{\theta}) = T_{\text{origin} \rightarrow \text{target}}[0:3, 0:3] \in \mathbb{R}^{3 \times 3} \quad (\text{rotation}), \quad (4)$$

$$\mathbf{r}(\boldsymbol{\theta}) = \text{vec}(R(\boldsymbol{\theta})) \in \mathbb{R}^9 \quad (\text{flattened rotation}). \quad (5)$$

Orientation is represented by the nine raw rotation-matrix entries rather than Euler angles or quaternions. This keeps the orientation residual linear in the matrix and avoids angle-wrap and parameterization singularities.

3 Jacobian (per sample)

The Jacobians of the predicted pose with respect to the parameters are computed *once* symbolically (`sympy`) and converted to fast numerical functions via `lambdify` (lines 213–233):

$$J_t(\boldsymbol{\theta}) = \frac{\partial \mathbf{t}}{\partial \boldsymbol{\theta}} \in \mathbb{R}^{3 \times 12}, \quad J_R(\boldsymbol{\theta}) = \frac{\partial \mathbf{r}}{\partial \boldsymbol{\theta}} \in \mathbb{R}^{9 \times 12}. \quad (6)$$

For sample k they are evaluated numerically at that sample’s commanded joint angles $\mathbf{q}^{(k)}$ (the current operating point) and measured pose $\mathbf{c}^{(k)}$ (lines 461–509), giving the numeric blocks $J_t^{(k)} \in \mathbb{R}^{3 \times 12}$ and $J_R^{(k)} \in \mathbb{R}^{9 \times 12}$.

4 Residual vector (per sample)

For each sample k the residual is the difference between the measured (ground-truth) pose and the model prediction (lines 513–548):

$$\Delta \mathbf{t}^{(k)} = \mathbf{t}_{\text{actual}}^{(k)} - \mathbf{t}_{\text{expected}}^{(k)} \in \mathbb{R}^3, \quad (7)$$

$$\Delta \mathbf{r}^{(k)} = \text{vec}(R_{\text{actual}}^{(k)} - R_{\text{expected}}^{(k)}) \in \mathbb{R}^9, \quad (8)$$

where “actual” is the camera-measured target pose and “expected” is the model prediction using the current parameter estimate.

5 Assembling the system matrix A and right-hand side $\mathbf{b}$

Let there be n samples in one iteration. The system is built in three steps, mirroring the code (`compute_jacobians`, lines 499–507, and `compute_calibration_parameters`, lines 569–573).

Step 1 — per-sample evaluation. For each sample $k = 1, \dots, n$, evaluate the translation and rotation Jacobian blocks and the two residual blocks:

$$J_t^{(k)} \in \mathbb{R}^{3 \times 12}, \quad J_R^{(k)} \in \mathbb{R}^{9 \times 12}, \quad \Delta \mathbf{t}^{(k)} \in \mathbb{R}^3, \quad \Delta \mathbf{r}^{(k)} \in \mathbb{R}^9.$$

Step 2 — vertical stacking by type. The per-sample blocks are stacked vertically (`np.vstack`), keeping all translation rows together and all rotation rows together:

$$\mathbf{J}_t = \begin{bmatrix} J_t^{(1)} \\ J_t^{(2)} \\ \vdots \\ J_t^{(n)} \end{bmatrix} \in \mathbb{R}^{3n \times 12}, \quad \mathbf{J}_R = \begin{bmatrix} J_R^{(1)} \\ J_R^{(2)} \\ \vdots \\ J_R^{(n)} \end{bmatrix} \in \mathbb{R}^{9n \times 12}, \quad (9)$$

$$\Delta \mathbf{t} = \begin{bmatrix} \Delta \mathbf{t}^{(1)} \\ \vdots \\ \Delta \mathbf{t}^{(n)} \end{bmatrix} \in \mathbb{R}^{3n}, \quad \Delta \mathbf{r} = \begin{bmatrix} \Delta \mathbf{r}^{(1)} \\ \vdots \\ \Delta \mathbf{r}^{(n)} \end{bmatrix} \in \mathbb{R}^{9n}. \quad (10)$$

The row ordering of $\mathbf{J}_t$ matches that of $\Delta \mathbf{t}$ sample by sample (and likewise for the rotation blocks), so each row of the stacked system is one scalar equation about one pose component of one sample.

Step 3 — weight and concatenate the two blocks. With translation weight $w_t = 1.0$ and rotation weight $w_R = 0.5$, the full system matrix and right-hand side are

$$A = \begin{bmatrix} w_t \mathbf{J}_t \\ w_R \mathbf{J}_R \end{bmatrix} \in \mathbb{R}^{12n \times 12}, \quad \mathbf{b} = \begin{bmatrix} w_t \Delta \mathbf{t} \\ w_R \Delta \mathbf{r} \end{bmatrix} \in \mathbb{R}^{12n}. \quad (11)$$

Written out fully, A is the $12n \times 12$ matrix

$$A = \begin{bmatrix} w_t J_t^{(1)} \\ \vdots \\ w_t J_t^{(n)} \\ w_R J_R^{(1)} \\ \vdots \\ w_R J_R^{(n)} \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} w_t \Delta \mathbf{t}^{(1)} \\ \vdots \\ w_t \Delta \mathbf{t}^{(n)} \\ w_R \Delta \mathbf{r}^{(1)} \\ \vdots \\ w_R \Delta \mathbf{r}^{(n)} \end{bmatrix}. \quad (12)$$

So A has $12n$ rows ($3n$ translation equations followed by $9n$ rotation equations) and 12 columns (one per parameter in $\boldsymbol{\theta}$).

6 Least-squares solution

With $12n \gg 12$ the stacked system is overdetermined and has no exact solution, so it is solved in the least-squares sense (equivalently, by minimizing $\|A\boldsymbol{\theta} - \mathbf{b}\|_2^2$) using the Moore–Penrose pseudo-inverse A^+ :

$$A\boldsymbol{\theta} = \mathbf{b} \quad \longrightarrow \quad \boldsymbol{\theta}^* = A^+ \mathbf{b} = (A^\top A)^{-1} A^\top \mathbf{b}. \quad (13)$$

In code this is computed with

```
errorEstimates, residuals, rank, singular_values = np.linalg.lstsq(AMat,
    bMat, rcond=None)
```

which forms the pseudo-inverse via singular value decomposition rather than the explicit normal-equation product $(A^\top A)^{-1} A^\top$. The SVD route is numerically stable under rank deficiency, and the returned `rank` and `singular_values` expose unobservable parameter combinations.

7 Damped, clipped, accumulated update

The raw least-squares solution is not applied in full. A relaxation factor and per-group magnitude caps are applied (lines 593–596):

```
updateCoefficient = 0.5
errorEstimates[0:6] = clip(0.5 * errorEstimates[0:6], -5, 5)    # dQ
errorEstimates[12:18] = clip(0.5 * errorEstimates[12:18], -5, 5) # dX
```

- **Half-step** ($\times 0.5$): a relaxed Gauss–Newton step. Because the forward kinematics is nonlinear and A is only a local linearization, a full step can overshoot; damping improves convergence stability.
- **Clipping** (± 5): rejects an outlier-driven wild step.
- The capped correction is split into ΔQ and ΔX ; the ΔX block is sign-flipped to match the model convention (line 620), and the running estimate is the cumulative sum over iterations (`np.cumsum`, line 638).

8 Iterative (Gauss–Newton) loop

Let j index the iteration and $\hat{\theta}_j$ the accumulated estimate. Each iteration (`process_iteration_results`, line 600):

1. Collect n measurements at poses commanded using $\hat{\theta}_j$ (the driver applies $q_{\text{cmd}} - \sum \Delta Q$, etc.).
2. Relinearize: form $\mathbf{b}$ and A at the new operating point (Section 5).
3. Solve $A\theta \approx \mathbf{b}$ by SVD least squares.
4. Damp ($\times 0.5$), clip (± 5), and accumulate: $\hat{\theta}_{j+1} = \hat{\theta}_j + 0.5\theta^*$.
5. Repeat for `numIters` iterations.

This is Gauss–Newton applied to the nonlinear least-squares problem $\min_{\theta} \sum_k \|\mathbf{m}_k - \mathbf{f}_k(\theta)\|^2$: each step linearizes $\mathbf{f}_k(\theta) = \text{FK}_k(\theta)$, takes a damped least-squares correction, and relinearizes, so the accumulated parameters converge toward the true ΔQ and ΔX .

Summary

Linearize $\text{pose} = \text{FK}(\theta)$ into $A = \partial \text{FK} / \partial \theta$; measure $\mathbf{b} = \text{pose}_{\text{measured}} - \text{pose}_{\text{predicted}}$; stack the per-sample blocks into the $12n \times 12$ system A and the $12n$ vector $\mathbf{b}$ (weighted translation rows then weighted flattened-rotation rows); solve $A\theta \approx \mathbf{b}$ by SVD least squares; apply a damped, clipped, accumulated step; and iterate.

Implementation note. `process_iteration_results` is called with (`measurements_actual=targetPoseMeasurements`, `measurements_expected=targetPoseExpected`) (line 600) but internally calls `compute_differences(measurements_expected, measurements_actual)` (line 604) — the actual/expected arguments are swapped at that boundary. The sign of $\mathbf{b}$ must remain consistent with the sign convention of A ; a flipped $\mathbf{b}$ would drive the Gauss–Newton step the wrong way (divergence rather than convergence). This is worth verifying.